QUANTUM SECURITY INTELLIGENCE

Security & Post-Quantum Readiness Audit Report

seedsigner @ ada81cde050e · SeedSigner — public repository (passive review)

Organization	QSI
Report date	2026-11-06 13:57 UTC
Report version	1
Audit run	87733b6d-a403-43e6-96bc-93e4e1806be1 · Document audit · 2026-11-06 13:57 UTC
Analyzer engine	qsi-engine 1.3.0
Risk profile	Crypto exchange, custodian or wallet
Content SHA-256	`01996796bcafb8fce4023132d4bba49a64437dd673f115989e79cf74109bfa43`
Signature key (Ed25519, SHA-256 fingerprint; the signature is the separate file report.sig.json)	`5b3cf9c9bd0fa54447e2b5dfa7a6bab569bd0c666fa46949599e5c348d17083e`

Contents

1. Executive summary
2. Scope and boundaries
3. Methodology
4. Assets examined and not examined
5. Architecture overview
6. Overall security posture
7. Findings by severity
8. Detailed findings
9. Evidence
10. Post-quantum assessment
11. Limitations and assumptions
12. Remediation plan
13. Priorities for 30, 60 and 90 days
14. Verification after remediation
15. Tools used
16. Sources and normative documents
17. Appendices

This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.

Executive summary

4	0	2	2	0
Findings in total	Critical	High	Medium	Low

Classification	Count
Confirmed technical vulnerability	0
Known vulnerable component	0
Confirmed configuration error	0
Potential cryptographic risk	2
Long-term post-quantum risk	2
Improvement recommendation	0
Insufficient data	0

Top 5

ID	Severity	Detailed findings	Priority score
QSI-2026-18989	High	Non-cryptographic generator next to key or seed generation in src/seedsigner/views/seed_views.py preliminary	38.8
QSI-2026-18990	High	Non-cryptographic generator next to key or seed generation in tools/mnemonic.py preliminary	38.8
QSI-2026-18987	Medium	Quantum-vulnerable RSA key (3072-bit) in seedsigner_pubkey.gpg	50.3
QSI-2026-18988	Medium	Quantum-vulnerable ECDSA: transaction and message signing (1 file(s)) confirmed by the analyst	41.8

Mosca's inequality (x + y > z): x = 10, y = 5, z = 8 → migration would finish too late under this scenario. z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Analyst review: 1 confirmed, 0 not applicable, 0 dismissed as false positives.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography. Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Scope and boundaries

Mode: Document audit

Repository: https://github.com/SeedSigner/seedsigner @ ada81cde050e46c19cb6e3fa73872b73955a138c — evidence links open the lines at this commit.

Supplied inputs

76 files, 990213 bytes in total, 0 redaction(s); the per-file SHA-256 list is in appendix C.

Methodology

The audit was performed by the QSI engine 1.3.0. Each analyzer emits facts (rule, parameters, evidence); finding texts are generated from the rule catalog. Methodology sources are listed under "Sources and normative documents".

Prioritization uses an explainable weighted model (severity, exploitability, exposure, confidentiality/integrity/availability impact weighted by the "Crypto exchange, custodian or wallet" profile, asset criticality, urgency, ease of remediation) with a confidence multiplier. CVSS v3.1 is applied only to technical vulnerabilities with a defined vector. Long-term quantum risks use Mosca's inequality; their severity is capped at Medium and they are never presented as exploitable vulnerabilities.

Finding classes: confirmed technical vulnerability; known vulnerable component (a dependency version named in an advisory; whether the affected code is reachable is assessed separately, and a component confirmed by review becomes a vulnerability); confirmed configuration error; potential cryptographic risk; long-term post-quantum risk; improvement recommendation; insufficient data. Heuristic findings are marked preliminary; analyst decisions are shown next to the findings.

Coverage of checks

Area	Status	Count	Examples
dependencies	checked	5	l10n/requirements-l10n.txt: 2 dependencies parsed (0 pinned); pyproject.toml: 0 dependencies parsed (0 pinned); requirements-raspi.txt: 4 dependencies parsed (4 pinned)
certificates_and_keys	checked	1	seedsigner_pubkey.gpg: certificates/keys parsed
source_code	checked	70	setup.py: 8 lines scanned with 16 rule patterns and 266 registry patterns; src/main.py: 53 lines scanned with 16 rule patterns and 266 registry patterns; src/seedsigner/__init__.py: 0 lines scanned with 16 rule patterns and 266 registry patterns
entropy	checked	1	random-number generation: 70 code files read with 0 macro definitions from the tree; 0 ineffective guard(s), 0 weak random-bytes provider(s), 2 weak key-generation site(s)
osv_advisories	checked	1	5 pinned packages queried at OSV.dev; 0 affected

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

Assets examined and not examined

Supplied inputs: 78 (input files: C)

Architecture overview

Static review of the source tree seedsigner at ada81cde050e46c19cb6e3fa73872b73955a138c. 76 files analysed; of 373 files, 70 are test, fuzzing, benchmark or example code and were left out, 226 are not read by any analyzer, 1 exceed the per-file limit and 0 did not fit the total limit. Files no analyzer reads include: .github/pull_request_template.md, .github/workflows/build.yml, .github/workflows/tests.yml, .gitignore, .gitmodules. 63 files in build-output or dependency-cache directories (.git, .hg, .venv, __pycache__, build, dist, node_modules, target, vendor, venv) were not read. 3 files come from directories a .gitmodules pins and are read as shipped source rather than skipped as a dependency cache: src/seedsigner/resources/seedsigner-translations (3). No requests were sent to the owner's systems.

Overall security posture

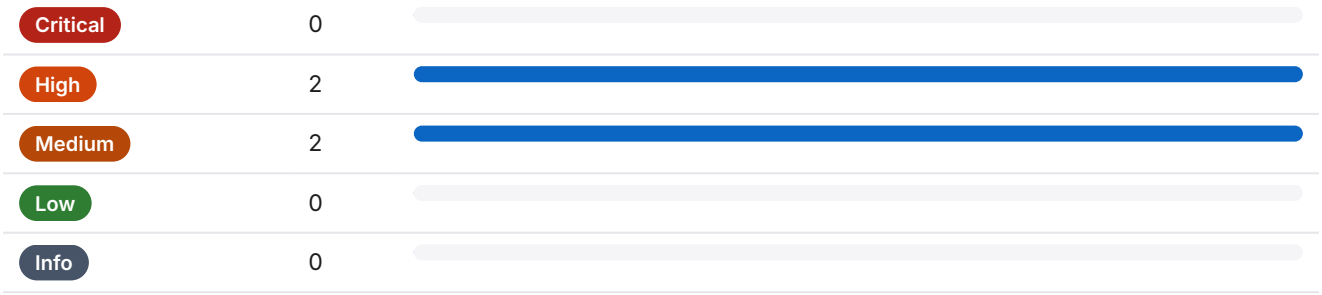
86
Indicative posture index (0–100)

33.3%
Quantum-vulnerable share of inventory

9
Cryptographic inventory

Derived from findings in scope (confirmed and potential issues weighted by severity). Indicative only; not comparable across differently scoped audits.

Findings by severity



ID	Detailed findings	Classification	Severity	Confidence
QSI-2026-18989	Non-cryptographic generator next to key or seed generation in src/seedsigner/views/seed_views.py	Potential cryptographic risk	High	Medium, preliminary
QSI-2026-18990	Non-cryptographic generator next to key or seed generation in tools/mnemonic.py	Potential cryptographic risk	High	Medium, preliminary
QSI-2026-18987	Quantum-vulnerable RSA key (3072-bit) in seedsigner_pubkey.gpg	Long-term post-quantum risk	Medium	High
QSI-2026-18988	Quantum-vulnerable ECDSA: transaction and message signing (1 file(s))	Long-term post-quantum risk	Medium	Medium

Detailed findings

QSI-2026-18989 · RNG-WEAK-KEYGEN

High

Non-cryptographic generator next to key or seed generation in src/seedsigner/views/seed_views.py

Potential cryptographic risk | Confidence: Medium | preliminary — requires further verification

'random.random(' at line 1284, within a few lines of 'mnemonic_list' (3 occurrence(s)).

Affected asset	src/seedsigner/views/seed_views.py:keygen
Data source	src/seedsigner/views/seed_views.py
Detection method	Regular-expression matching of language-specific API patterns.
Analyzer	entropy 1.3.0 · 2026-11-06 13:57 UTC
Priority score	38.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable

POTENTIAL IMPACT

Mnemonics, seeds or private keys drawn from such a generator can be enumerated from its small internal state: Milk Sad (CVE-2023-39910, mt19937 behind 'bx seed') and Trust Wallet (CVE-2023-31290) lost funds this way.

REPRODUCTION STEPS

```
Open line 1284 in the supplied file src/seedsigner/views/seed_views.py and inspect the quoted lines
(see evidence).
```

EVIDENCE

file_excerpt · src/seedsigner/views/seed_views.py · line 1284 · SHA-256 d723de33db421cb8… · File 4c977bac0dbf3e89…

```
 1282 |
 1283 |         real_word = ButtonOptionWithoutTranslation(self.mnemonic_list[self.cur_index])
 1284 |         fake_word1 = ButtonOptionWithoutTranslation(bip39.WORDLIST[int(random.random() *
2047)])
 1285 |         fake_word2 = ButtonOptionWithoutTranslation(bip39.WORDLIST[int(random.random() *
2047)])
 1286 |         fake_word3 = ButtonOptionWithoutTranslation(bip39.WORDLIST[int(random.random() *
2047)])
```

file_excerpt · src/seedsigner/views/seed_views.py · line 1285 · SHA-256 caf93e3f449ccd2d… · File 4c977bac0dbf3e89…

```
 1283 |         real_word = ButtonOptionWithoutTranslation(self.mnemonic_list[self.cur_index])
 1284 |         fake_word1 = ButtonOptionWithoutTranslation(bip39.WORDLIST[int(random.random() *
2047)])
 1285 |         fake_word2 = ButtonOptionWithoutTranslation(bip39.WORDLIST[int(random.random() *
2047)])
 1286 |         fake_word3 = ButtonOptionWithoutTranslation(bip39.WORDLIST[int(random.random() *
2047)])
 1287 |
```

file_excerpt · src/seedsigner/views/seed_views.py · line 1286 · SHA-256 4ecc0b33ab779205… · File 4c977bac0dbf3e89…

```
 1284 |         fake_word1 = ButtonOptionWithoutTranslation(bip39.WORDLIST[int(random.random() *
2047)])
 1285 |         fake_word2 = ButtonOptionWithoutTranslation(bip39.WORDLIST[int(random.random() *
2047)])
 1286 |         fake_word3 = ButtonOptionWithoutTranslation(bip39.WORDLIST[int(random.random() *
```

```
2047)])
 1287 |
 1288 |         button_data = [real_word, fake_word1, fake_word2, fake_word3]
```

LIMITATIONS OF THE CHECK

Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Use the platform CSPRNG (secrets, os.urandom, crypto.getRandomValues, crypto/rand, SecureRandom, OsRng) or a hardware TRNG for every key, seed and mnemonic.

RETEST PROCEDURE

Re-scan the source and review the call sites listed in the evidence.

CWE-338 — Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG); CVE-2023-39910 — Libbitcoin Explorer 'bx seed' entropy (Milk Sad): mt19937 limits it to 32 bits; CVE-2023-31290 — Trust Wallet Core: 32 bits of mnemonic entropy (mt19937 seeded with one 32-bit value)

QSI-2026-18990 · RNG-WEAK-KEYGEN

High

Non-cryptographic generator next to key or seed generation in tools/mnemonic.py

Potential cryptographic risk | Confidence: Medium | preliminary — requires further verification

'random.randint(' at line 84, within a few lines of 'mnemonic_generation' (4 occurrence(s)).

Affected asset	tools/mnemonic.py:keygen
Data source	tools/mnemonic.py
Detection method	Regular-expression matching of language-specific API patterns.
Analyzer	entropy 1.3.0 · 2026-11-06 13:57 UTC
Priority score	38.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable

POTENTIAL IMPACT

Mnemonics, seeds or private keys drawn from such a generator can be enumerated from its small internal state: Milk Sad (CVE-2023-39910, mt19937 behind 'bx seed') and Trust Wallet (CVE-2023-31290) lost funds this way.

REPRODUCTION STEPS

```
Open line 84 in the supplied file tools/mnemonic.py and inspect the quoted lines (see evidence).
```

EVIDENCE

file_excerpt · tools/mnemonic.py · line 84 · SHA-256 81554da16391deb1... · File 92d3059fc0eb8624...

```
   82 |     if method == 'dice':
   83 |         if zero_indexed_dice:
   84 |             entropy = ''.join([str(random.randint(0, 5)) for i in
range(mnemonic_generation.DICE__NUM_ROLLS__12WORD if mnemonic_length == 12 else
mnemonic_generation.DICE__NUM_ROLLS__24WORD)])
   85 |         else:
   86 |             entropy = ''.join([str(random.randint(1, 6)) for i in
```

```
range(mnemonic_generation.DICE__NUM_ROLLS__12WORD if mnemonic_length == 12 else
mnemonic_generation.DICE__NUM_ROLLS__24WORD)])
```

file_excerpt · tools/mnemonic.py · line 86 · SHA-256 1a73f1c331308a7e... · File 92d3059fc0eb8624...

```
   84 |             entropy = ''.join([str(random.randint(0, 5)) for i in
range(mnemonic_generation.DICE__NUM_ROLLS__12WORD if mnemonic_length == 12 else
mnemonic_generation.DICE__NUM_ROLLS__24WORD)])
   85 |         else:
   86 |             entropy = ''.join([str(random.randint(1, 6)) for i in
range(mnemonic_generation.DICE__NUM_ROLLS__12WORD if mnemonic_length == 12 else
mnemonic_generation.DICE__NUM_ROLLS__24WORD)])
   87 |
   88 |     elif method == 'coins':
```

file_excerpt · tools/mnemonic.py · line 89 · SHA-256 c781ae848a2442a4... · File 92d3059fc0eb8624...

```
   87 |
   88 |     elif method == 'coins':
   89 |         entropy = ''.join([str(random.randint(0, 1)) for i in range(128 if mnemonic_length
== 12 else 256)])
   90 |
   91 |     elif method == 'final_word':
```

LIMITATIONS OF THE CHECK

Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Use the platform CSPRNG (secrets, os.urandom, crypto.getRandomValues, crypto/rand, SecureRandom, OsRng) or a hardware TRNG for every key, seed and mnemonic.

RETEST PROCEDURE

Re-scan the source and review the call sites listed in the evidence.

CWE-338 — Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG); CVE-2023-39910 — Libbitcoin Explorer 'bx seed' entropy (Milk Sad): mt19937 limits it to 32 bits; CVE-2023-31290 — Trust Wallet Core: 32 bits of mnemonic entropy (mt19937 seeded with one 32-bit value)

QSI-2026-18987 · KEY-QV-PUBLIC

Medium

Quantum-vulnerable RSA key (3072-bit) in seedsigner_pubkey.gpg

Long-term post-quantum risk | Confidence: High

A RSA 3072-bit public key was found (line 1).

Affected asset	file:seedsigner_pubkey.gpg#line 1
Data source	seedsigner_pubkey.gpg
Detection method	Public key parsing.
Analyzer	certificates 1.3.0 · 2026-11-06 13:57 UTC
Priority score	50.3 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk once a CRQC exists; for key exchange, recorded traffic could be decrypted retroactively.

REPRODUCTION STEPS

```
Open line 1 in the supplied file seedsigner_pubkey.gpg and inspect the quoted lines (see evidence).
```

EVIDENCE

key · seedsigner_pubkey.gpg · line 1 · SHA-256 315636b2f7e00661… · File cb7d694d3c7f3c65…

```
OpenPGP primary key: algorithm ID 1 (RSA) 3072-bit
```

key · seedsigner_pubkey.gpg · line 1 · SHA-256 6ccbdce52e43463e… · File cb7d694d3c7f3c65…

```
OpenPGP subkey: algorithm ID 1 (RSA) 3072-bit
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability.

RECOMMENDATION

Inventory and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it..

RETEST PROCEDURE

Re-audit after migration.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards

QSI-2026-18988 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDSA: transaction and message signing (1 file(s))

Long-term post-quantum risk | Confidence: Medium

Analyst review (QSI review, 2026-10-29): confirmed by the analyst. Correct, and it is the only row of its kind because it is the only place SeedSigner touches the primitive itself: src/seedsigner/helpers/embit_utils.py:10, the wrapper around embit. Everything that signs is embit's, and embit is not in this tree. Three consequences worth stating rather than leaving implied. First, the honest reading of "1 Shor-breakable row" is "this repository delegates its signing", not "this device is nearly post-quantum". Second, the pin is exact and hash-locked (embit==0.8.0 with a sha256 --hash), so the audited exposure is a specific, reproducible artefact rather than a version range — which is what makes the delegation auditable at all, and is the opposite of a criticism. Third, the migration question for SeedSigner is therefore not "when will SeedSigner adopt a post-quantum signature" but "when will embit", and after that, "how fast can a hash-locked pin be moved" — which is a release away, not a redesign. Compare Krux, which pins the same library as a git submodule and so carries the same code

> inside its own tree: 98 rows and 64 Shor-breakable for the same cryptography, differing only in where it is kept.

3 reference(s) to ECDSA across 1 file(s), e.g. src/seedsigner/helpers/embit_utils.py:10. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

Affected asset	source-code:ECDSA:transaction_signing
Data source	src/seedsigner/helpers/embit_utils.py
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-06 13:57 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open src/seedsigner/helpers/embit_utils.py:10 and inspect the quoted lines; the evidence lists
further files.
```

EVIDENCE

file_excerpt · src/seedsigner/helpers/embit_utils.py · line 10 · SHA-256 1e5604518dfe3ecc… · File c087d19323033176…

```
   9 | from embit.networks import NETWORKS
  10 | from embit.util import secp256k1
  11 |
```

file_excerpt · src/seedsigner/helpers/embit_utils.py · line 204 · SHA-256 1060384364a8c023… · File c087d19323033176…

```
 203 |     prv = root.derive(derivation).key
 204 |     sig = secp256k1.ecdsa_sign_recoverable(msghash, prv._secret)
 205 |     flag = sig[64]
```

file_excerpt · src/seedsigner/helpers/embit_utils.py · line 209 · SHA-256 1e4a07630105e8ba… · File c087d19323033176…

```
 208 |     flag = bytes([27 + flag + c])
 209 |     ser = flag + secp256k1.ecdsa_signature_serialize_compact(sig._sig)
 210 |     return b2a_base64(ser).strip().decode()
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

Evidence

ID	Type	Data source	SHA-256
QSI-2026-18989	file_excerpt	src/seedsigner/views/seed_views.py (line 1284)	d723de33db421cb83fece9c3e89e0ebba4d5ddc972fb2ae1e8abe90202b94
QSI-2026-18989	file_excerpt	src/seedsigner/views/seed_views.py (line 1285)	caf93e3f449ccd2d13b25280a33505adffb00dc1fa21c1dc79755cf6ace9c
QSI-2026-18989	file_excerpt	src/seedsigner/views/seed_views.py (line 1286)	4ecc0b33ab779205a0bf0403f83a00dd861f9c240c5b9828147db3879f501
QSI-2026-18990	file_excerpt	tools/mnemonic.py (line 84)	81554da16391deb1865464a4df4ebb0a2b7e3b8ad2020319c7c6e0dc836c
QSI-2026-18990	file_excerpt	tools/mnemonic.py (line 86)	1a73f1c331308a7e40d04099df3a3eed3986396f4951026ee1a73b191c8a1
QSI-2026-18990	file_excerpt	tools/mnemonic.py (line 89)	c781ae848a2442a4d4da68c06d6cebc1b94de618065507f33f9975fe6d87a
QSI-2026-18987	key	seedsigner_pubkey.gpg (line 1)	315636b2f7e00661811d8d8feee53223db00c9277ae1194dcaf9d4fb25bf0
QSI-2026-18987	key	seedsigner_pubkey.gpg (line 1)	6ccbdce52e43463e8b282489a66bb4553923cec099cc342f213f584e97c81
QSI-2026-18988	file_excerpt	src/seedsigner/helpers/embit_utils.py (line 10)	1e5604518dfe3ecc40df631b9df1ab8a9e5e838974ac1c814502658edc7d1
QSI-2026-18988	file_excerpt	src/seedsigner/helpers/embit_utils.py (line 204)	1060384364a8c023d525064551aeed7ece6fd577f80394a1a55de0cce5980
QSI-2026-18988	file_excerpt	src/seedsigner/helpers/embit_utils.py (line 209)	1e4a07630105e8ba280ad6a4b597896533871c3bfe4f6dc7bdfcc27b6efb1

Post-quantum assessment

	Key establishment (x = how long data must stay confidential; harvest now, decrypt later)	Signatures (x = how long signatures/ keys must stay trusted)
Data/key lifetime x	10	10
Migration time y	5	5
Assumed years to CRQC z	8 (early)	8
x + y − z	7 → migration would finish too late under this scenario	7 → migration would finish too late under this scenario

Planning assumptions: x for key establishment, x for signatures, y are the defaults of the risk profile “Crypto exchange, custodian or wallet”, not the client's figures; with the client's actual lifetimes the verdict can change.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography

Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Levels: 0 none; 1 present, not wired; 2 in use outside the critical paths; 3 hybrid in a critical path; 4 post-quantum by default in the critical paths (root of trust, key wrapping, channel key exchange). Judged from code and manifests.

What the quantum-vulnerable algorithms do

Role	Algorithms (files)	ID	First step
transaction and message signing	ECDSA (1)	QSI-2026-18988	Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them

Roles are judged from file paths, function names and the code around each match; review confirms them. Root-of-trust keys and wrapped keys come first: they are hard to replace or can be recorded today.

z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Quantum status	Count
reduced margin (Grover), approved	2
Grover: acceptable to NIST, below ECCG/ANSSI guidance	4
vulnerable (Shor)	3

Detailed findings: QSI-2026-18987, QSI-2026-18988

PQC readiness questionnaire

No readiness questionnaire was completed for this project.

Quantum algorithms relevant to this inventory

Quantum algorithm	Effect	Inventoried algorithms (rows)	What it means for this audit
Shor's algorithm — factoring Q1: quantum computation offline, classical access to the victim	breaks	RSA (2)	Breaks RSA at every key length: a longer modulus only raises the cost polynomially. Plan migration to ML-KEM (or a hybrid) for key establishment and ML-DSA / SLH-DSA for signatures; start with keys protecting data that must stay confidential for years (harvest now, decrypt later) and with long-lived signatures.
Shor's algorithm — elliptic-curve discrete logarithms Q1: quantum computation offline, classical access to the victim	breaks	ECDSA (1)	Elliptic-curve keys are expected to fall before RSA keys of comparable classical strength. On blockchains, coins whose public key is already on chain (reused addresses, pay-to-public-key and Taproot outputs) are exposed first; plan the move to post-quantum signatures and avoid key reuse in the meantime.
Grover's algorithm — key search and preimages Q1: quantum computation offline, classical access to the victim	weakens (larger keys restore the margin)	PBKDF2 (1), SHA-2 (5)	Not a break. 256-bit keys (AES-256, ChaCha20) keep a comfortable margin; AES-128 meets NIST category 1, while some authorities ask for 256-bit keys in quantum-resistant contexts (see the AES-128 entry). Prefer 256-bit keys for long-lived data; 256-bit hashes need no change for preimage resistance.
Brassard–Høyer–Tapp — collision search Q1: quantum computation offline, classical access to the victim	weakens (larger keys restore the margin)	SHA-2 (5)	256-bit hashes (SHA-256, SHA3-256) remain adequate for collision resistance; 160-bit hashes are already weak classically (SHA-1 collisions are practical). Use 384/512-bit hashes where NIST categories 4–5 are required.

Effects: breaks — replace the algorithm; weakens — larger keys or outputs restore the margin; included in standard parameters — no action at standard parameter sets; only with superposition queries — not applicable to ordinary deployments.

Limitations and assumptions

- This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.
- z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Remediation plan

QSI-2026-18989 · P1 · 30 days

root-cause fix

Use the platform CSPRNG (secrets, os.urandom, crypto.getRandomValues, crypto/rand, SecureRandom, OsRng) or a hardware TRNG for every key, seed and mnemonic.

Rationale	Mnemonics, seeds or private keys drawn from such a generator can be enumerated from its small internal state: Milk Sad (CVE-2023-39910, mt19937 behind 'bx seed') and Trust Wallet (CVE-2023-31290) lost funds this way.
Expected outcome	Re-scan the source and review the call sites listed in the evidence.
Complexity	medium
Tests before change	Open line 1284 in the supplied file src/seedsigner/views/seed_views.py and inspect the quoted lines (see evidence).
Tests after change	Re-scan the source and review the call sites listed in the evidence.
Closure criteria	Re-scan the source and review the call sites listed in the evidence.

QSI-2026-18990 · P1 · 30 days

root-cause fix

Use the platform CSPRNG (secrets, os.urandom, crypto.getRandomValues, crypto/rand, SecureRandom, OsRng) or a hardware TRNG for every key, seed and mnemonic.

Rationale	Mnemonics, seeds or private keys drawn from such a generator can be enumerated from its small internal state: Milk Sad (CVE-2023-39910, mt19937 behind 'bx seed') and Trust Wallet (CVE-2023-31290) lost funds this way.
Expected outcome	Re-scan the source and review the call sites listed in the evidence.
Complexity	medium
Tests before change	Open line 84 in the supplied file tools/mnemonic.py and inspect the quoted lines (see evidence).
Tests after change	Re-scan the source and review the call sites listed in the evidence.
Closure criteria	Re-scan the source and review the call sites listed in the evidence.

QSI-2026-18987 · P3 · 90 days

short-term risk reduction

Add RSA usages at file:seedsigner_pubkey.gpg to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18988 · P3 · 90 days

short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18987 · P4 · 90 days

root-cause fix

Plan migration of RSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it. (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it. or a hybrid scheme in use.

QSI-2026-18988 · P4 · 90 days

root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

Priorities for 30, 60 and 90 days

30 days

Priority	ID	Action	Type
P1	QSI-2026-18989	Use the platform CSPRNG (secrets, os.urandom, crypto.getRandomValues, crypto/rand, SecureRandom, OsRng) or a hardware TRNG for every key, seed and mnemonic.	root-cause fix
P1	QSI-2026-18990	Use the platform CSPRNG (secrets, os.urandom, crypto.getRandomValues, crypto/rand, SecureRandom, OsRng) or a hardware TRNG for every key, seed and mnemonic.	root-cause fix

60 days

Priority	ID	Action	Type
None			

90 days

Priority	ID	Action	Type
P3	QSI-2026-18987	Add RSA usages at file:seedsigner_pubkey.gpg to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-18988	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P4	QSI-2026-18987	Plan migration of RSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it. (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-18988	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix

Verification after remediation

ID	Retest procedure
QSI-2026-18989	Re-scan the source and review the call sites listed in the evidence.
QSI-2026-18990	Re-scan the source and review the call sites listed in the evidence.
QSI-2026-18987	Re-audit after migration.
QSI-2026-18988	Re-run the inventory after refactoring.

Tools used

Tool	Version
blockchain	1.0.0
certificates	1.2.0
chain_map	1.0.0
configs	1.1.0
dependencies	1.3.0
inventory_document	1.1.0
solidity	1.3.0
source_code	1.4.0
tls_code	1.1.0
tls_live	1.0.0
qsi-engine	1.3.0
OSV.dev API	v1

Sources and normative documents

Reviewed: 2026-10-13

1. A. Chailloux, M. Naya-Plasencia, A. Schrottenloher — An Efficient Quantum Collision Search Algorithm and Implications on Symmetric Cryptography (ASIACRYPT 2017; IACR ePrint 2017/847) — IACR ePrint. https://eprint.iacr.org/2017/847 (peer-reviewed article: Õ(2^{2n/5}) time, O(n) qubits, Õ(2^{n/5}) classical memory)
2. C. Chevignard, P.-A. Fouque, A. Schrottenloher — Reducing the Number of Qubits in Quantum Discrete Logarithms on Elliptic Curves (IACR ePrint 2026/280) — IACR ePrint. https://eprint.iacr.org/2026/280 (revision of a EUROCRYPT 2026 paper: 3.12n + o(n) qubits; for n = 256, 1193 qubits and 22 runs of 2^38.98 Toffoli gates)
3. C. Gidney — How to factor 2048 bit RSA integers with less than a million noisy qubits (arXiv:2505.15917) — arXiv. https://arxiv.org/abs/2505.15917 (preprint (2025-05-21): < 1 million noisy qubits, < 1 week, 0.1% gate error, 1 µs surface-code cycle)
4. C. Gidney, M. Ekerå — How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits (Quantum 5, 433) — Quantum. https://doi.org/10.22331/q-2021-04-15-433 (peer-reviewed article (2021): 10^-3 gate error, 1 µs cycle, 10 µs reaction time)
5. C. Zalka — Grover's quantum searching algorithm is optimal (Phys. Rev. A 60, 2746, 1999; arXiv:quant-ph/9711070) — APS. https://doi.org/10.1103/PhysRevA.60.2746 (peer-reviewed article: optimal for any success probability; parallel search gains no more than splitting the space)
6. CVE-2023-31290 — Trust Wallet Core: 32 bits of mnemonic entropy (mt19937 seeded with one 32-bit value) — NIST NVD. https://nvd.nist.gov/vuln/detail/CVE-2023-31290 (published 2023-04-27; Trust Wallet Core before 3.1.1 (browser extension 0.0.172-0.0.182), exploited in the wild in December 2022 and March 2023: only about four billion possible mnemonics)
7. CVE-2023-39910 — Libbitcoin Explorer 'bx seed' entropy (Milk Sad): mt19937 limits it to 32 bits — NIST NVD. https://nvd.nist.gov/vuln/detail/CVE-2023-39910 (published 2023-08-09; Libbitcoin Explorer 3.0.0-3.6.0, exploited in the wild in 2023)
8. CWE-338 — Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG) — MITRE. https://cwe.mitre.org/data/definitions/338.html (weakness catalogue entry: the consequence is predictable values where unpredictability was assumed, which is what makes a token or key guessable)
9. Common Vulnerability Scoring System v3.1: Specification Document — FIRST. https://www.first.org/cvss/v3.1/specification-document (final)
10. D. J. Bernstein — Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete? (2009) — cr.yp.to. https://cr.yp.to/hash/collisioncost-20090823.pdf (paper dated 2009-08-23)
11. D. Litinski — How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates (arXiv:2306.08585) — arXiv. https://arxiv.org/abs/2306.08585 (preprint (2023-06-14): 6000 modules × 1152 physical qubits, one key per 10 minutes under its architecture assumptions)
12. FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM) — NIST. https://csrc.nist.gov/pubs/fips/203/final (final (August 2024))
13. FIPS 204 — Module-Lattice-Based Digital Signature Standard (ML-DSA) — NIST. https://csrc.nist.gov/pubs/fips/204/final (final (August 2024))
14. FIPS 205 — Stateless Hash-Based Digital Signature Standard (SLH-DSA) — NIST. https://csrc.nist.gov/pubs/fips/205/final (final (August 2024))
15. G. Brassard, P. Høyer, A. Tapp — Quantum Algorithm for the Collision Problem (LATIN '98; arXiv:quant-ph/9705002) — Springer / arXiv. https://arxiv.org/abs/quant-ph/9705002 (peer-reviewed article: collisions of r-to-one functions in O((N/r)^{1/3}) expected evaluations)
16. J. Proos, C. Zalka — Shor's discrete logarithm quantum algorithm for elliptic curves (Quantum Inf. Comput. 3(4), 2003; arXiv:quant-ph/0301141) — Quantum Information & Computation. https://doi.org/10.26421/QIC3.4-3 (peer-reviewed article)
17. L. K. Grover — A fast quantum mechanical algorithm for database search (STOC 1996) — ACM. https://doi.org/10.1145/237814.237866 (peer-reviewed article)
18. M. Amy et al. — Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3 (SAC 2016; arXiv:1603.09383) — Springer / arXiv. https://arxiv.org/abs/1603.09383 (peer-reviewed article: SHA-256 about 2^153.8 surface-code cycles on 2^12.6 logical qubits; SHA3-256 about 2^146.5 cycles on 2^20)
19. M. Cain et al. — Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits (arXiv:2603.28627) — arXiv. https://arxiv.org/abs/2603.28627 (preprint (2026-03-30): P-256 in a few days with 26,000 physical neutral-atom qubits; RSA-2048 one to two orders of magnitude longer)
20. M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018) — IEEE. https://doi.org/10.1109/MSP.2018.3761723 (peer-reviewed article; source of the x + y > z migration inequality)

21. M. Roetteler, M. Naehrig, K. Svore, K. Lauter — Quantum resource estimates for computing elliptic curve discrete logarithms (ASIACRYPT 2017) — IACR / arXiv. https://arxiv.org/abs/1706.06752 (peer-reviewed article: at most 9n + 2⌈log2 n⌉ + 10 qubits and 448 n^3 log2 n + 4090 n^3 Toffoli gates)
22. NIST IR 8547 — Transition to Post-Quantum Cryptography Standards — NIST. https://csrc.nist.gov/pubs/ir/8547/final (final (2026-10-27; the initial public draft of November 2024 is obsoleted). Tables 2 and 4: ECDSA, EdDSA and RSA signatures, finite-field and elliptic-curve DH and MQV, and RSA key establishment are deprecated after 2030 and disallowed after 2035 at every key size — the draft deprecated only the 112-bit level. A composite (hybrid) key establishment stays FIPS 140-validatable while one input comes from an approved scheme. Repeats the federal deadlines of OMB M-26-15: post-quantum key establishment by 2030-12-31 and signatures by 2031-12-31 for high value assets and high impact systems)
23. NIST — Post-Quantum Cryptography Standardization: Evaluation Criteria, Security (Call for Proposals, Section 4.A.5) — NIST. https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization/Evaluation-Criteria/Security-(Evaluation-Criteria) (security categories: AES-128 key search 2^170/MAXDEPTH quantum gates, AES-192 2^233/MAXDEPTH, AES-256 2^298/MAXDEPTH; MAXDEPTH 2^40–2^96)
24. P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997) — SIAM. https://doi.org/10.1137/S0097539795293172 (peer-reviewed article)
25. R. Babbush et al. — Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations (arXiv:2603.28846) — arXiv. https://arxiv.org/abs/2603.28846 (preprint (2026-03-30, v2 2026-04-15): 256-bit ECDLP with < 1200 logical qubits and < 90 M Toffoli gates (or < 1450 and < 70 M); < 500,000 physical qubits at 10^-3 error, minutes)
26. S. Jaques, M. Naehrig, M. Roetteler, F. Virdia — Implementing Grover oracles for quantum key search on AES and LowMC (EUROCRYPT 2020; arXiv:1910.01700) — Springer / arXiv. https://arxiv.org/abs/1910.01700 (peer-reviewed article: lower cost estimates for the AES key searches that define the NIST categories)

Appendices

A. Cryptographic inventory

Algorithm	Parameter	Location	Quantum status	Classical status
PBKDF2		src/seedsigner/models/seed.py:186	reduced margin (Grover), approved	acceptable
SHA-2	SHA-512	src/seedsigner/models/seed.py:181	reduced margin (Grover), approved	acceptable
SHA-2	SHA-256	src/seedsigner/gui/screens/tools_screens.py:112	Grover: acceptable to NIST, below ECCG/ANSSI guidance	acceptable
SHA-2	SHA-256	src/seedsigner/helpers/mnemonic_generation.py:74	Grover: acceptable to NIST, below ECCG/ANSSI guidance	acceptable
SHA-2	SHA-256	src/seedsigner/helpers/ur2/xoshiro256.py:66	Grover: acceptable to NIST, below ECCG/ANSSI guidance	acceptable
SHA-2	SHA-256	src/seedsigner/views/tools_views.py:183	Grover: acceptable to NIST, below ECCG/ANSSI guidance	acceptable
ECDSA	secp256k1	src/seedsigner/helpers/embit_utils.py:10	vulnerable (Shor)	acceptable
RSA	3072-bit	seedsigner_pubkey.gpg (line 1)	vulnerable (Shor)	acceptable
RSA	3072-bit	seedsigner_pubkey.gpg (line 1)	vulnerable (Shor)	acceptable

B. JSON

qsi.report/1 · SHA-256 01996796bcafb8fce4023132d4bba49a64437dd673f115989e79cf74109bfa43

C. Input files

File	Size	SHA-256	Rec
l10n/requirements-l10n.txt	1099	`9ffcfaafedd10eee5b8020d7a9c6117e00eebaa354aa2248a0b1f7974f809cc3`	0
pyproject.toml	2006	`806812aaa643b69646850af44736db858192095ae1da0dd6432fa9a52a2a9331`	0
requirements-raspi.txt	98	`97a87553c8c5e02b39450fd57707fef2d68d1564c89402d3d25be468eb3c8481`	0
requirements.txt	7698	`f43fdf89d5cfe76e4f1d01d27df6c5030d6aa1767217d9d8c855e4a5fe34b45a`	0
seedsigner_pubkey.gpg	2456	`cb7d694d3c7f3c655d9d2e7a30a45b60fd6d8fb30e37afc53440eb156df9e57a`	0
setup.py	181	`d79d8ecf1aa4f4e496d725ea43fa2082d91016d760ad1581aae0ca8e5be61c4b`	0
src/main.py	1540	`0fec0abc13bf4f8d754e78ee16649b51028a85c3286f2abcb860fcc0b723bf27`	0
src/seedsigner/__init__.py	0	`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`	0
src/seedsigner/controller.py	18529	`2b536116eb906cb6b30ef1ee0aeb85ff3b6e5dd4cb49f30ce9cf55ede4cbb97d`	0
src/seedsigner/gui/__init__.py	30	`dbcabb7d019bd42ebf2861fff87f66cbd004a652b0f5ea912505bc3b4dbe3258`	0

src/seedsigner/gui/components.py	79525	9bf81d18f771279eaf856a536a59a54d9b445fb7fe777ce1a0dbf412e844af93	0
src/seedsigner/gui/keyboard.py	25548	dbfa96f48aec0b0d6cf7d83f4b493aca81fa4f67fee4d7c11e310c622e393334	0
src/seedsigner/gui/renderer.py	4501	308c77a3e190df410245c7c2762e2ee7464fa73933d6da4e8acc2a936460af67	0
src/seedsigner/gui/screens/__init__.py	21	a5496c911fdee0f9907e2d37723907443f11db5d81eb9559a8a527aba51a4b9f	0
src/seedsigner/gui/screens/psbt_screens.py	33501	d8fae1241725686353d884bf816325bcff936d64ad831700234ecc943354063b	0
src/seedsigner/gui/screens/scan_screens.py	13062	938d846d7c518a035ab0b757fd87ee2c75cbe64b00ed5959e42592f35d460696	0
src/seedsigner/gui/screens/screen.py	55000	0212127e2533cdc72d421fb0ebc58f8f3278f1d3a4960ba9941abb01051f8eff	0
src/seedsigner/gui/screens/seed_screens.py	70769	c0b9eca6e4c76947997ba1352d3971ac259391f4746b19d74ba24d89bf85b17a	0
src/seedsigner/gui/screens/settings_screens.py	19504	e379674269d2edcce5f794a41b05896f4b692db39d5e19e3ea774415232c9a49	0
src/seedsigner/gui/screens/tools_screens.py	31245	4e407f8bfd50fe12fcd6e32037b44895abea4c2d196df6bdcaf228839bd03da4	0
src/seedsigner/gui/toast.py	13196	ec4cb0206b3d3d9903be66dfedee0bf920ade3bb71fb1f8496de8988bab7b245	0
src/seedsigner/hardware/__init__.py	0	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855	0
src/seedsigner/hardware/buttons.py	7931	4471efdfa2dfd502ab17f1a0e016bacb1031060293f9fb27c00d988748f625f7	0
src/seedsigner/hardware/camera.py	3241	4d1d61508ab5f83ebe23a20e2f9ea01fbd9269f3026865720d515497f717533b	0
src/seedsigner/hardware/displays/ST7789.py	4997	4d43e4974a43bb8aa12f7e0697ade09894a582085dfe67eec6b9582930479a7e	0
src/seedsigner/hardware/displays/__init__.py	0	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855	0
src/seedsigner/hardware/displays/display_driver.py	3068	99b99244032d27ce88865a1e4e5d3765e933a9b46a97e8f4ac63e4b05ca6a2d8	0
src/seedsigner/hardware/displays/ili9341.py	12574	c73eb77327bb882ecc78e7e5af355af969ce09d22435d981168cdab017fad998	0
src/seedsigner/hardware/displays/st7789_mpy.py	32972	71347193cdc17edb2aa54485f38befaa827dcadc02687d826d32f1597a3ce89f	0
src/seedsigner/hardware/microsd.py	2389	6b7dfe146c69b151eb4fe3dad3bd4e518875f04e30d51ec544d4d6432d317ac1	0
src/seedsigner/hardware/pivideostream.py	1819	0eb5e7d8333ed7d590d79eecdadf395d1c71d77772febea413dd1f924909e51c	0
src/seedsigner/helpers/__init__.py	0	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855	0
src/seedsigner/helpers/embit_utils.py	7727	c087d1932303317654685fd24b3753429964a5b4f63c5d927f1663bc9b18c3d1	0
src/seedsigner/helpers/l10n.py	162	611cbe5643158179649 99f3437eecc6a988e2b968a8169066b146e0d1f784686	0
src/seedsigner/helpers/mnemonic_generation.py	4630	1da00ac502a88ae77733681fd19f0fb61df03ebdee759b9ac78798fe9399c8fc	0

src/seedsigner/helpers/qr.py	4808	ed12c8c01002e52af9946e2c0b579c4fd692dee81612dabbc77e73fb1f0bdca8	0
src/seedsigner/helpers/ur2/__init__.py	0	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855	0
src/seedsigner/helpers/ur2/bytewords.py	4929	b652f89758fb84d2beafaeb01e77a8af5f3e7a201d5b3fa1473e01ef8418bfc1	0
src/seedsigner/helpers/ur2/cbor_lite.py	11208	313c06f8d23542fcd7eaa0c25ccb57d568571ba7afd6e630c5d9d418c2265913	0
src/seedsigner/helpers/ur2/constants.py	179	d7a4ccd6c624c69aec6983667399c6566dddb60134349e43443ac05b67956495	0
src/seedsigner/helpers/ur2/crc32.py	1037	888541118b69e5a3f587e1556f0787f6b644ff263811c6913843eb18b68513da	0
src/seedsigner/helpers/ur2/fountain_decoder.py	12114	4285d087cb3dcfb740c9c92329c4ba805cc3044f1a874fb0daa33405ef4fec3f	0
src/seedsigner/helpers/ur2/fountain_encoder.py	5244	155812c2f58fc22888b6b2ec601d40fe405f6a4cbc0be58794c8052bf411b25a	0
src/seedsigner/helpers/ur2/fountain_utils.py	1597	cbbc87708a1ad1f4e5494f04a4c29ca585946fa55fb0a4dd92711d7d40c20d42	0
src/seedsigner/helpers/ur2/random_sampler.py	1568	dcb23762815edd2a0b0fe48fba02fac5b3aba3d67c1f27bafe6823b1dda04b59	0
src/seedsigner/helpers/ur2/ur.py	494	b51393572a556ff4426412783b1902d1afc6241f26870ecf649eefb719880929	0
src/seedsigner/helpers/ur2/ur_decoder.py	5203	61fc0d5319f6915aab118bcd13e6aa25e0e021c5b4dcc6af2222ff30a14e9c6a	0
src/seedsigner/helpers/ur2/ur_encoder.py	2430	2a05145e5961347404ccf7230727eee85efa2822f0b39890ae42e6fe47202c1b	0
src/seedsigner/helpers/ur2/utils.py	1619	565724921749e1ed0e92b6beb38d4f00df2ec373b105928bed1f8c30da297c6d	0
src/seedsigner/helpers/ur2/xoshiro256.py	4397	d4d700a789502a4ba21f4a3a24a3c8381f13a3c7fb896b0e19dffc6948aef1d4	0
src/seedsigner/helpers/version.py	37956	70759ff9f528343912572b8ff4e1597cf08f4371029346e93b5875322ed3b7b7	0
src/seedsigner/models/__init__.py	0	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855	0
src/seedsigner/models/decode_qr.py	39285	ca35d03c9ede201893a01e5faceb96d58449e1600a28e564ae29d6a2b0248912	0
src/seedsigner/models/encode_qr.py	13142	cba66b93b19ed0831e27bec44c49e6e488cf1634f2b82e5c5f3e9cffe9695957	0
src/seedsigner/models/psbt_parser.py	82783	3f8cbba4d378aa4f82ac9a1d4c05b5b7017f9b720eb92c38d530f4aab3dafbe3	0
src/seedsigner/models/qr_type.py	930	56b4789b1bc55e2b129d0a1fe873f7b2ff2d7ddafe62f8fce6c0e4d3814b5230	0
src/seedsigner/models/seed.py	7768	4f612eeae59dd972bd9187818c6648a70efe7e258f4d5c81662f9175060a7658	0
src/seedsigner/models/seed_storage.py	2957	2c2ad94d462898ca0d83b7dc935750db96d60c173aa0422dffc41ca9b2cde686	0
src/seedsigner/models/settings.py	13398	864853cc67cd59d60c220bceb922c502fe06db3993de3de76acee11604e23c13	0
src/seedsigner/models/settings_definition.py	35542	ffdefda767a713a8c7a54dffd712739daa235b01efdffa2068fbaba86b241cce	0

src/seedsigner/models/singleton.py	1059	caaa41a60af389c8b5322d510536d310664fa4ef53e8957c9fab7f021c4e71ac	0
src/seedsigner/models/threads.py	1079	678a61d601a2c8a0d5ff3b1c110f781b07b17a2f152e20a6fcf55e95f6a531ee	0
src/seedsigner/views/__init__.py	50	07d90e6e3c9bc4127568b08e5bdf2725ac093964a987c50d52efe3e4dca8d505	0
src/seedsigner/views/psbt_views.py	41251	47fff120f41e2710226560d722b70d51a8bee0ece389b9b3567092988b413750	0
src/seedsigner/views/scan_views.py	9526	f49166b5417dbc649da3b2b7871308aae608f573312e105945de36568d3628d3	0
src/seedsigner/views/screensaver.py	9727	e3eb0f54536dcd7d02296c5e5c6aa34ffcae152e24e994cc67d1e4b626a9d3cc	0
src/seedsigner/views/seed_views.py	90475	4c977bac0dbf3e893b0e58b7fabdf0ab89b141f32717e038de659a097e470d4c	0
src/seedsigner/views/settings_views.py	15828	eaaa53d1250f6ba624cbddb0a696306f08187653a598f783002e51a5c3c09797	0
src/seedsigner/views/tools_views.py	34242	c9dde008d0e3565a27bb572529d6a9a23607fc9cfa1560e6d4b12ab68e11f537	0
src/seedsigner/views/view.py	16231	b92cb1e521001a270d678ef7d32126c7736fecd629f781251a860a4d349d957d	0
tools/mnemonic.py	6971	92d3059fc0eb86241d4d8839d6c64116423291526d093bb4f1b37fdd95c13f60	0
tools/seed_phrase_to_qr.py	1580	de71bf293c28d5c4e637c545a07685160c2aa92c9f64213d7a5259d9e91de66b	0
tools/write_versionfile.py	1602	2bb499c921b6f8cf04f40f835ca3a6cd92ea445c50deb893e90d38262e19f53a	0
src/seedsigner/resources/seedsigner-translations/.github/diff_report/diff_screenshots.py	6279	7a1ecba23853b008f72921dfb453b14959e35ed7d8f0ebfa906a4eb0908319f2	0
src/seedsigner/resources/seedsigner-translations/tools/extract_characters_from_babel_mo.py	2700	df7055b2f0c8460ee5fd9f1e995cb7a8346e43af628d90a2b08b46d7766cd7ff	0
src/seedsigner/resources/seedsigner-translations/tools/requirements.txt	6	68359c38ed3bf77f71bad8d21c219fbe8c094d44374b35d2974e202f8b24472b	0